

Building the Tree, Not Just Walking It

Expression Tree Construction · A Priority Queue Bridge

Unit III • Lecture 24

Lecture 21 introduced expression trees. Lecture 23 formalized WALKING them. Today: the missing piece, BUILDING one from postfix — and a short bridge to priority queues.

The idea: postfix evaluation, but with tree nodes

- 1** **Operand:** Make a leaf node. Push it.
- 2** **Operator:** Pop two nodes (right first, then left). Make a new node with the operator as root and those two as its children. Push the new node.
- 3** **End of input:** Exactly one node remains on the stack — that is the root of the whole tree.

buildExpressionTree(): the algorithm in C

```

struct Node *buildExpressionTree(
    const char postfix[])
{
    top = -1;

    for (int i = 0; postfix[i]; i++)
    {
        char ch = postfix[i];
        if (ch == ' ') continue;

        if (isOperator(ch))
        {
            struct Node *right = pop();
            struct Node *left = pop();
            struct Node *node = newNode(ch);
            node->left = left;
            node->right = right;
            push(node);
        }
        else
            push(newNode(ch));
    }
    return pop();
}

```

- The stack here holds struct Node *, not int — the only structural change from Lecture 10.
- isOperator(ch) checks for + - * /.
- The final pop() returns the root — the whole tree, in one pointer.

Trace: building from $A B C * + D -$ (verified)

Sym	Action	Stack (bottom→top)
A	push leaf	A
B	push leaf	A B
C	push leaf	A B C
*	pop C, pop B $\rightarrow *(B, C)$, push	A *
+	pop *, pop A $\rightarrow +(A, *)$, push	+
D	push leaf	+ D
-	pop D, pop + $\rightarrow -(+, D)$, push	-

One node remains: the '-' built at step 7. It is the root — exactly the tree from Lectures 21 and 23.

The round trip: build, then walk, and check

```
struct Node *root = buildExpressionTree("A B C * + D -");
```

```
preorder(root) = - + A * B C D    (prefix)
```

```
inorder(root) = A + B * C - D    (infix)
```

```
postorder(root) = A B C * + D -    (SAME as the input!)
```

Bridge: why priority queues sit next to trees here

Priority queues were built in full back in Unit II, Lecture 14 — insert, removeHighest, the $O(1)$ -insert/ $O(n)$ -remove trade-off, and the promise that a better structure was coming.

That promised structure is a HEAP — and a heap is a complete BINARY TREE.

- Lecture 22 already gave you the array representation a heap needs — index i 's children at $2i+1$, $2i+2$.
- A heap keeps the tree SHAPED so the highest priority is always sitting at the root — removeHighest becomes $O(\log n)$, insert too.
- Full heap construction and operations are covered properly in Unit IV, Lecture 35 — today just names the connection.

Construction is evaluation's mirror image

- buildExpressionTree scans postfix with a stack of NODES: operand -> push leaf; operator -> pop two, combine, push.
- The same pop-order trap as Lecture 10 applies: right pops first, left pops second.
- One node remains at the end — the root. Construction and postorder traversal are inverses of each other on the same tree.
- Priority queues (Lecture 14) and heaps (Lecture 35) both rest on the tree ideas built across this unit — shape, array representation, and now construction.

EXIT TICKET Build the expression tree for postfix $P\ Q +\ R\ S -\ *$ by hand, tracing the node stack after every symbol. What is the root?